

App Dev Project Report

1. Student Details

Name: Pratosh Lathia

Roll Number: 23f3003255

Email: 23f3003255@ds.study.iitm.ac.in

About Me: I am a student with an interest in full-stack web development

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

Institutes require efficient systems to manage campus recruitment processes involving students, companies, and placement drives.

Objective:

To build a centralized Placement Portal that enables structured interaction between Admin, Company, and Student users with proper approval workflows and application tracking.

Approach:

The application is developed using Flask with SQLAlchemy ORM and follows a role-based access system using sessions.

3. AI/LLM Declaration

Bootstrap/CSS (~8%), Implementing Download Resume, DB Searching Logic, Debugging, Improving application logic, make code clean i.e proper code structure in project files (app.py and html files)

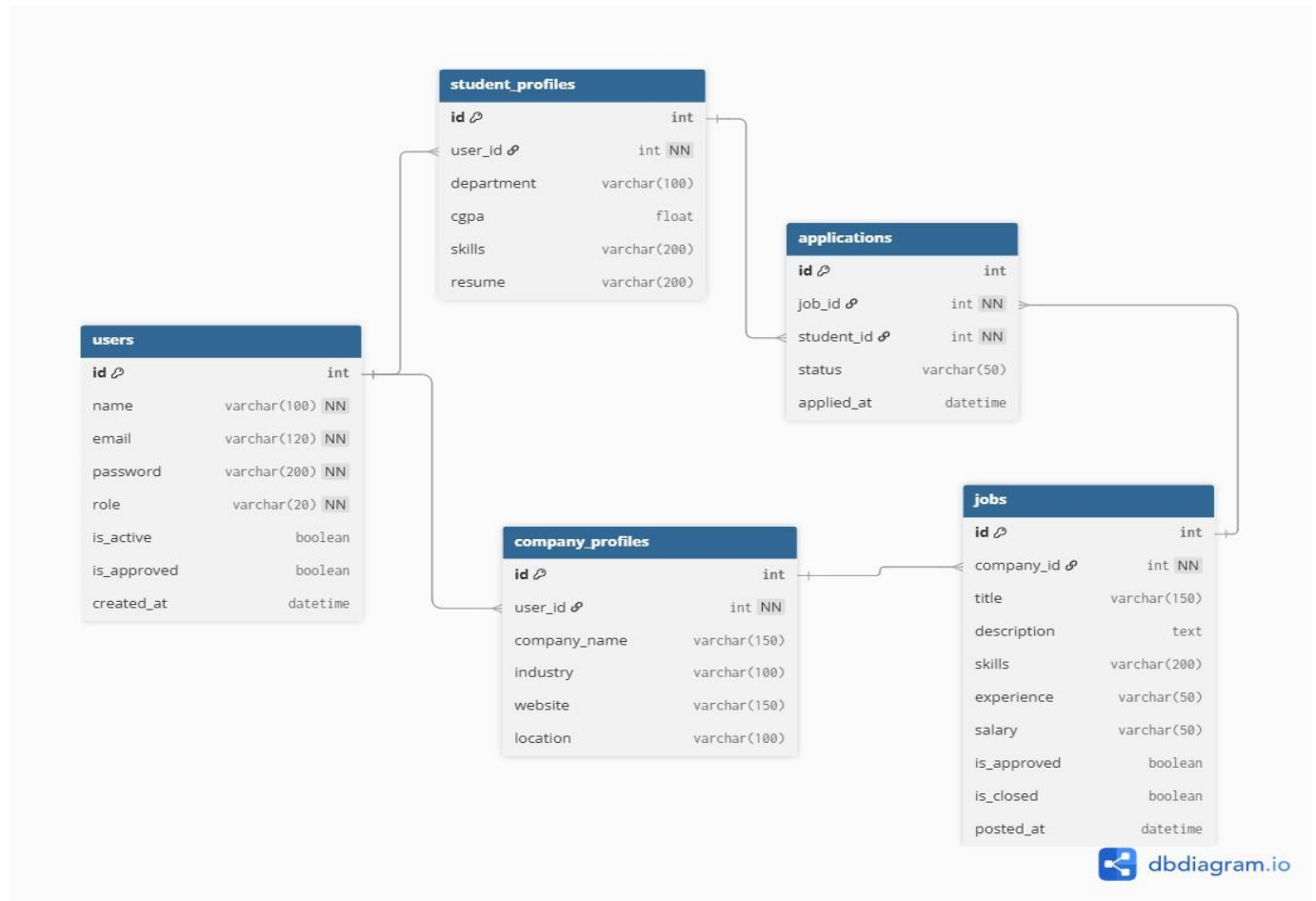
The usage was limited to approximately **10-20%**

All core implementation, database design, and integration were performed manually.

4. Technologies and Frameworks Used

Python	Programming language
HTML/CSS	Frontend structure
Bootstrap 5	Frontend styling and responsive design
Flask	Core backend web framework
SQLite	Lightweight local database for storing user data
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Flask-Login	User authentication and session management
Werkzeug	Uploading resume to defined path

5. Database Schema / ER Diagram



User

- *id* (PK)
- *name*
- *email* (unique)
- *password*
- *role* (admin / student / company)
- *is_active*
- *is_approved*
- *created_at*

StudentProfile

- *id (PK)*
- *user_id (FK, unique)*
- *department*
- *cgpa*
- *skills*
- *resume*

CompanyProfile

- *id (PK)*
- *user_id (FK, unique)*
- *company_name*
- *industry*
- *website*
- *location*

Job

- *id (PK)*
- *company_id (FK)*
- *title*
- *description*
- *skills*
- *experience*
- *salary*
- *is_approved*
- *is_closed*
- *posted_at*

Application (Association Table)

- *id (PK)*
- *job_id (FK)*
- *student_id (FK)*
- *status (Applied / Shortlisted / Selected / Rejected)*
- *applied_at*

Relationships

- User → StudentProfile → One-to-One
- User → CompanyProfile → One-to-One
- CompanyProfile → Job → One-to-Many

Many-to-Many Relationship (Important)

- **Student ↔ Job → Many-to-Many (via Application table)**

StudentProfile → Application → One-to-Many

Job → Application → One-to-Many

A student can apply to multiple jobs, and a job can have multiple students applying. This Many-to-Many relationship is implemented using the **Application table**, which acts as an **association table** and stores additional attributes like status and applied date.

2. API Resource Endpoints (Complete)

Authentication

- / → Home page
- /register → Register user
- /login → Login
- /logout → Logout

Company

- /company_dashboard → Dashboard
- /company_profile → Create/update profile
- /post-job → Create job
- /edit-job/<job_id> → Edit job
- /delete-job/<job_id> → Delete job (safe check)
- /confirm-delete/<job_id> → Force delete
- /toggle-job/<job_id> → Open/close job
- /company/student/<student_id> → View student details
- /job-applications/<job_id> → View applications

Student

- `/student_dashboard` → View jobs + search
- `/student_profile` → Create/update profile
- `/apply-job/<job_id>` → Apply to job
- `/my_applications/<student_id>` → View applications

Application Management

- `/toggle-status/<app_id>/<action>` → Update status
 - shortlist
 - select
 - Reject

Admin

- `/admin_dashboard` → Dashboard
- `/admin/user/<user_id>/toggle` → Activate/deactivate
- `/admin/company/<user_id>` → View company
- `/admin/job/<job_id>/applications` → View applications
- `/admin/company/<user_id>/approve` → Approve company
- `/admin/company/<user_id>/reject` → Reject company
- `/admin/company/<user_id>/blacklist` → Blacklist
- `/admin/company/<user_id>/unblacklist` → Unblacklist
- `/admin/job/<job_id>/approve` → Approve job
- `/admin/job/<job_id>/reject` → Reject job
- `/admin/student/<user_id>` → View student
- `/admin/student/<user_id>/blacklist` → Blacklist student
- `/admin/student/<user_id>/unblacklist` → Unblacklist student

6. Architecture and Features

Architecture Overview

- app.py → Main application (routes + models)
- Templates → Jinja2 HTML pages
- Static → CSS, Bootstrap, JS
- Database → SQLite

Core Features

Admin

- Approves/rejects companies
- Approves/rejects job postings
- Manages students and companies
- Search functionality
- Blacklisting system

Company

- Profile creation and update
- Job posting and editing
- Close/open job postings
- View applicants
- Manage application status

Student

- Profile creation
- Job search (company/title/skills)
- Apply for jobs
- Track application status

User Interface Design

- Built using **Bootstrap 5** for responsive design
- Uses components like:
 - Navbar
 - Cards
 - Forms
 - Buttons
- Ensures:
 - Mobile-friendly layout
 - Clean and consistent UI
 - Improved user experience

Advanced Features

- Search functionality using filters and joins
- Duplicate application prevention
- Multi-stage application status system
- Safe job deletion mechanism
- Approval workflows (company & job)
- Role-based access control using sessions
- Dashboard statistics for admin
- Responsive UI using Bootstrap

Limitations

- Passwords stored in plain text
- No email notifications
- No REST API support

Future Enhancements

- Implement password hashing
- Email notifications
- REST API development
- Advanced frontend (React/AJAX)